# SIMATS ENGINEERING

## Short Answer Test

**COURSE NAME: Software Testing** **COURSE CODE: CSA37**

**COURSE OUTCOME COVERED**

**CO1:** *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

| Sl. No | Questions | Bloom's Level |
|---|---|---|
| 1 | A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection. | 3 |
| 2 | A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples. | 3 |
| 3 | During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes. | 3 |
| 4 | A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples. | 3 |
| 5 | A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example. | 3 |
| 6 | During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement. | 3 |
| 7 | A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues. | 3 |

| | | |
|---|---|---|
| 8 | A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue. | 3 |
| 9 | A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality. | 3 |
| 10 | An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented. | 3 |
| 11 | A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency. | 3 |
| 12 | A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution. | 3 |
| 13 | A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency. | 3 |
| 14 | During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps. | 3 |
| 15 | A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality. | 3 |
| 16 | A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability. | 3 |
| 17 | A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality. | 3 |
| 18 | An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled. | 3 |

| | | |
|---|---|---|
| 19 | A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability. | 3 |
| 20 | A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design. | 3 |
| 21 | A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency. | 3 |
| 22 | A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality. | 3 |
| 23 | A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures. | 3 |
| 24 | A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis. | 3 |
| 25 | A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability. | 3 |
| 26 | A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection. | 3 |
| 27 | A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes. | 3 |
| 28 | A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness. | 3 |
| 29 | A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage. | 3 |
| 30 | A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency. | 3 |

***Code of Conduct:***

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

## RUBRICS FOR CALCULATION:

| Criteria | Weightage | Level 4 – Exemplary (5 Marks) | Level 3 – Proficient (4 Marks) | Level 2 – Developing (2–3 Marks) | Level 1 – Beginning (0–1 Mark) |
|---|---|---|---|---|---|
| **Conceptual Understanding** | **30** | Demonstrates clear and complete understanding of the concept | Good understanding with minor gaps | Basic understanding with some misconceptions | Poor or incorrect understanding |
| **Accuracy of Answer** | **25** | Completely correct answer with no errors | Mostly correct with minor errors | Partially correct with noticeable errors | Incorrect or irrelevant answer |
| **Application of Knowledge** | **20** | Correctly applies concepts to solve the question/problem | Applies concepts with minor mistakes | Limited or partially correct application | No or incorrect application |
| **Completeness of Response** | **15** | Covers all required parts of the question | Covers most parts with minor omissions | Covers only some parts | Incomplete or missing response |
| **Clarity & Presentation** | **10** | Clear, concise, and well-organized answer | Understandable with minor issues | Somewhat unclear or poorly structured | Difficult to understand |

DATE: 15/10/26. CO1 - AT#2

SOFTWARE TESTING.

CSA3714.

NAME: LOGA DHARSHANA. A

REG NO: 192312130.

4,8,10,11,15,17,22,27,29,30.

4) · Error: A human mistake during requirements analysis, design or coding.

Defect: A flaw in the software caused by an error.

Failure: Incorrect software behavior observed during execution.

Human error → Software defect → Possible failure.

example: A developer misunderstands password requirement (error). writes incorrect validation logic (defect) and the application accepts an invalid password (failure).

8) · A boundary-related defect occurs when software handles minimum or maximum input values incorrectly.

* Boundary value analysis test the values at and around the given input range.

eg: if the age limit of a webs is from 18 to 60.

test 17, 18, 59, 60, 61, 19.

*. These tests help identify incorrect range checks and input-handling errors.

10) A functional defect occurs when software does not behave according to its specified requirements.

* In an ATM, deducting money without dispensing cash is

an example of incorrect transaction behavior.

Test cases should cover:

1. Successful withdrawal.
2. Invalid PIN.
3. Insufficient account balance.
4. Min and max withdrawal limit.
5. Network interrupts during withdrawals.
6. Correct balance updates

The tester should report the defect, verify the fix and perform regression testing.

---

11) Defect clustering is the principle that a large proportion of software defects are often concentrated in a small no. of modules.

* The testing team should analyse historical defect records to identify defect-prone modules and prioritize them according to risk and business impact.

---

15. Defect prevention aims to eliminate the root causes of software defects rather than repeatedly fixing their symptoms.

The team should:

1. Conduct root-cause analysis.
2. Review requirements for ambiguity.
3. Improve coding standards and code reviews.
4. Maintain a defect repository.
5. Add previously discovered defects to regression testing.

22. Tester should participate throughout the Software Development Life Cycle (SDLC):

1. Requirement analysis.
2. Design
3. Development
4. Testing
5. Maintenance

Early testing reduces rework, improves communication and helps identify defects before implementation.

27. Process-based testing follows defined procedures for test-planning, test-case design, execution, defect reporting, retesting and closure.

The team should:

1. Establish common testing procedures
2. Use standard test case and defect report templates.
3. Maintain consistent defect classifications
4. Track requirements and test coverage.
5. Conduct reviews and regression testing.

29. Defect leakage occurs when defect escape testing and are discovered after software deployment.

To reduce leakage, the team should:

1. Review requirements early.
2. Prioritize high-risk functions.
3. Test valid, invalid & boundary inputs.
4. Perform integration & system testing.
5. Analyse production defects and their root causes

30. Early defect detection means identifying defects during requirement analysis, design and development rather than waiting until deployment.

The team should:

1. Review requirements for ambiguity.
2. Involve testers during the early SDLC stages.
3. Prepare test cases before execution.
4. Perform unit and integration testing.
5. Track defects and find their root causes.